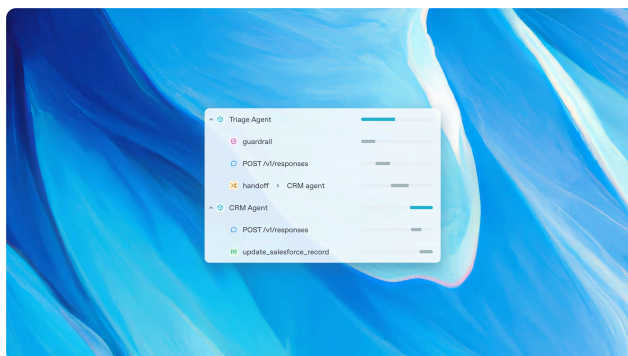


New tools for building agents

OpenAI · 2025-03-11 · 原文 / Source



Today, we're releasing the first set of building blocks that will help developers and enterprises build useful and reliable agents. We view agents as systems that independently accomplish tasks on behalf of users. Over the past year, we've introduced new model capabilities—such as advanced reasoning, multimodal interactions, and new safety techniques—that have laid the foundation for our models to handle the complex, multi-step tasks required to build agents. However, customers have shared that turning these capabilities into production-ready agents can be challenging, often requiring extensive prompt iteration and custom orchestration logic without sufficient visibility or built-in support.

To address these challenges, we're launching a new set of APIs and tools specifically designed to simplify the development of agentic applications:

- The new [Responses API](#), combining the simplicity of the Chat Completions API with the tool use capabilities of the Assistants API for building agents
- Built-in tools including [web search](#), [file search](#), and [computer use](#)
- The new [Agents SDK](#) to orchestrate single-agent and multi-agent workflows
- Integrated [observability tools](#) to trace and inspect agent workflow execution

These new tools streamline core agent logic, orchestration, and interactions, making it significantly easier for developers to get started with building agents. Over the coming weeks and months, we plan to release additional tools and capabilities to further simplify and accelerate building agentic applications on our platform.

Introducing the Responses API

The Responses API is our new API primitive for leveraging OpenAI's built-in tools to build agents. It combines the simplicity of Chat Completions with the tool-use capabilities of the Assistants API. As model capabilities continue to evolve, we believe the Responses API will provide a more flexible foundation for developers building agentic applications. With a single Responses API call, developers will be able to solve increasingly complex tasks using multiple tools and model turns.

To start, the Responses API will support new built-in tools like web search, file search, and computer use. These tools are designed to work together to connect models to the real world, making them more useful in completing tasks. It also brings with it several usability improvements including a unified item-based design, simpler polymorphism, intuitive streaming events, and SDK helpers like

`response.output_text` to easily access the model's text output.

The Responses API is designed for developers who want to easily combine OpenAI models and built-in tools into their apps, without the complexity of integrating multiple APIs or external vendors. The API also makes it easier to store data on OpenAI so developers can evaluate agent performance using features such as tracing and evaluations. As a reminder, we do not train our models on business data by default, even when the data is stored on OpenAI. The API is available to all developers starting today and is not charged separately—tokens and tools are billed at standard rates specified on our [pricing page](#). Check out the Responses API [quickstart guide](#) to learn more.

What this means for existing APIs

- [Chat Completions API](#): Chat Completions remains our most widely adopted API, and we're fully committed to supporting it with new models and capabilities. Developers who don't require built-in tools can confidently continue using Chat Completions. We'll keep releasing new models to Chat Completions whenever their capabilities don't depend on built-in tools or multiple model calls. However, the Responses API is a superset of Chat Completions with the same great performance, so for new integrations, we recommend starting with the Responses API.
- [Assistants API](#): Based on developer feedback from the Assistants API beta, we've incorporated key improvements into the Responses API, making it more flexible, faster, and easier to use. We're working to achieve full feature parity between the Assistants and the Responses API, including support for Assistant-like and Thread-like objects, and the Code Interpreter tool. Once this is complete, we plan to formally announce the deprecation of the Assistants API with a target sunset date in mid-2026. Upon deprecation, we will provide a clear migration guide from the Assistants API to the Responses API that allows developers to preserve all their data and migrate their applications. Until we formally announce the deprecation, we will continue delivering new models to the Assistants API. The Responses API represents the future direction for building agents on OpenAI.

Introducing built-in tools in the Responses API

Web search

Developers can now get fast, up-to-date answers with clear and relevant citations from the web. In the Responses API, web search is available as a tool when using gpt-4o and gpt-4o-mini, and can be paired with other tools or function calls.

JavaScript

```
const response = await openai.responses.create({
  model: "gpt-4o",
  tools: [ { type: "web_search_preview" } ],
  input: "What was a positive news story that happened today?",
});

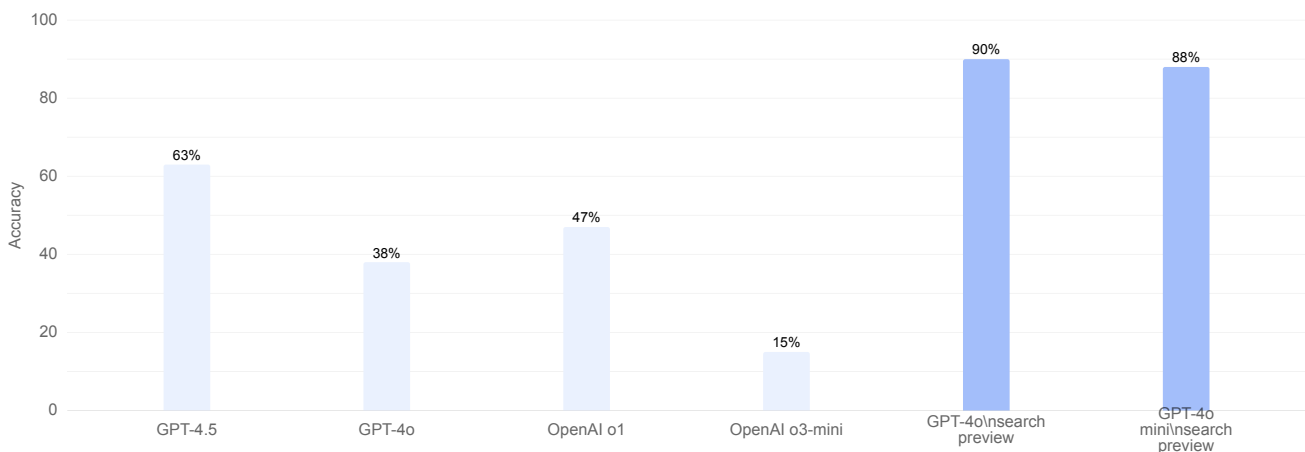
console.log(response.output_text);
```

During early testing, we've seen developers build with web search for a variety of use cases including shopping assistants, research agents, and travel booking agents—any application that requires timely information from the web.

For example, [Hebbia](#) leverages the web search tool to help asset managers, private equity and credit firms, and law practices quickly extract actionable insights from extensive public and private datasets. By integrating real-time search capabilities into their research workflows, Hebbia delivers richer, context-specific market intelligence and continuously improves the precision and relevance of their analyses, outperforming current benchmarks.

Web search in the API is powered by the same model used for ChatGPT search. On SimpleQA, a benchmark that evaluates the accuracy of LLMs in answering short, factual questions, GPT-4o search preview and GPT-4o mini search preview score 90% and 88% respectively.

SimpleQA Accuracy (higher is better)



Responses generated with web search in the API include links to sources, such as news articles and blog posts, giving users a way to learn more. With these clear, inline citations, users can engage with information in a new way, while content owners gain new opportunities to reach a broader audience.

Any website or publisher can [choose to appear](#) in web search in the API.

The web search tool is available to all developers in preview in the Responses API. We are also giving developers direct access to our fine-tuned search models in the Chat Completions API via `gpt-4o-search-preview` and `gpt-4o-mini-search-preview`. [Pricing](#) starts respectively at \$30 and \$25 per thousand queries for GPT-4o search and 4o-mini search respectively. Check out web search in the [Playground](#) and learn more in our [docs](#).

File search

Developers can now easily retrieve relevant information from large volumes of documents using the improved file search tool. With support for multiple file types, query optimization, metadata filtering, and custom reranking, it can deliver fast, accurate search results. And again, with the Responses API, it takes only a few lines of code to integrate.

JavaScript

```
const productDocs = await openai.vectorStores.create({
  name: "Product Documentation",
  file_ids: [file1.id, file2.id, file3.id],
});

const response = await openai.responses.create({
  model: "gpt-4o-mini",
  tools: [{
    type: "file_search",
    vector_store_ids: [productDocs.id],
  }],
  input: "What is deep research by OpenAI?",
});

console.log(response.output_text);
```

The file search tool can be used for a variety of real-world use cases, including enabling a customer support agent to easily access FAQs, helping a legal assistant to quickly reference past cases for a qualified professional, and assisting a coding agent to query technical documentation. For example, [Navan](#) uses file search in its AI-powered travel agent to quickly provide their users with precise answers from knowledge-base articles (like their company's travel policy). With built-in query optimization and reranking, they are able to set up a powerful RAG (retrieval-augmented generation) pipeline without extra tuning or configuration. With dedicated vector stores for each user group, Navan is able to tailor answers to individual account settings and user roles, saving time for customers and their staff while helping provide accurate, personalized support.

This tool is available in the Responses API to all developers. Usage is [priced](#) at \$2.50 per thousand queries and file storage at \$0.10/GB/day, with the first GB free. The tool continues to be available in the Assistants API. Finally, we've also added a new search endpoint to Vector Store API objects to directly query your data for use in other applications and APIs. Learn more in our [docs](#) and start testing in the [Playground](#).

Computer use

To build agents capable of completing tasks on a computer, developers can now use the computer use tool in the Responses API, powered by the same Computer-Using Agent (CUA) model that enables Operator. This research preview model set a new state-of-the-art record, achieving 38.1% success on OSWorld for full computer use tasks, 58.1% on WebArena, and 87% on WebVoyager for web-based interactions.

The built-in computer use tool captures mouse and keyboard actions generated by the model, making it possible for developers to automate computer use tasks by directly translating these actions into executable commands within their environments.

JavaScript

```
const response = await openai.responses.create({
  model: "computer-use-preview",
  tools: [{
    type: "computer_use_preview",
    display_width: 1024,
    display_height: 768,
    environment: "browser",
  }],
  truncation: "auto",
  input: "I'm looking for a new camera. Help me find the best one.",
});

console.log(response.output);
```

Developers can use the computer use tool to automate browser-based workflows like performing quality assurance on web apps or executing data-entry tasks across legacy systems. For example, Unify is a system of action for growing revenue that uses agents to identify intent, research accounts, and engage with buyers. Using OpenAI's computer use tool, Unify's agents can access information that was previously unreachable via APIs—such as enabling a property management company to verify through online maps if a business has expanded its real estate footprint. This research acts as a custom signal to trigger personalized outreach—empowering go-to-market teams to engage buyers with precision and scale.

As another example, Luminai integrated the computer use tool to automate complex operational workflows for large enterprises with legacy systems that lack API availability and standardized data. In a recent pilot with a major community service organization, Luminai automated the application processing and user enrollment process in just days—something traditional robotic process automation (RPA) struggled to achieve after months of effort.

Before launching CUA in Operator last year, we conducted extensive safety testing and red teaming, addressing three key areas of risk: misuse, model errors, and frontier risks. To address risks associated with expanding Operator's capabilities to local operating systems through CUA in the API, we performed additional safety evaluations and red teaming. We also added mitigations for developers, including safety checks to guard against prompt injections, confirmation prompts for sensitive tasks,

tools to help developers isolate their environments, and enhanced detection of potential policy violations. While these mitigations help reduce risk, the model is still susceptible to inadvertent mistakes, especially in non-browser environments. For example, CUA’s performance on OSWorld, a benchmark designed to measure the performance of AI agents on real-world tasks, is currently at 38.1%, indicating that the model is not yet highly reliable for automating tasks on operating systems. Human oversight is recommended in these scenarios. More details about our API-specific safety work can be found in our updated [system card](#).

Benchmark type	Benchmark	Computer use (universal interface)		Web browsing agents	Human
		OpenAI CUA	Previous SOTA	Previous SOTA	
Computer use	OSWorld	38.1%	22.0%	-	72.4%
Browser use	WebArena	58.1%	36.2%	57.1%	78.2%
	WebVoyager	87.0%	56.0%	87.0%	-

Evaluation details are described [here](#)

Starting today, the computer use tool is available as a research preview in the Responses API for select developers in [usage tiers 3-5](#). Usage is [priced](#) at \$3/1M input tokens and \$12/1M output tokens. Learn more in our [docs](#) and check out the [sample application](#) illustrating how to build with this tool.

Agents SDK

In addition to building the core logic of agents and giving them access to tools so they are useful, developers also need to orchestrate agentic workflows. Our new open-source Agents SDK simplifies orchestrating multi-agent workflows and offers significant improvements over [Swarm](#), an experimental SDK we released last year that was widely adopted by the developer community and successfully deployed by multiple customers.

Improvements include:

- **Agents:** Easily configurable LLMs with clear instructions and built-in tools.
- **Handoffs:** Intelligently transfer control between agents.
- **Guardrails:** Configurable safety checks for input and output validation.
- **Tracing & Observability:** Visualize agent execution traces to debug and optimize performance.

Python

```
from agents import Agent, Runner, WebSearchTool, function_tool, guardrail

@function_tool
def submit_refund_request(item_id: str, reason: str):
    # Your refund logic goes here
    return "success"

support_agent = Agent(
    name="Support & Returns",
    instructions="You are a support agent who can submit refunds [...]",
    tools=[submit_refund_request],
)

shopping_agent = Agent(
    name="Shopping Assistant",
    instructions="You are a shopping assistant who can search the web [...]",
    tools=[WebSearchTool()],
)

triage_agent = Agent(
    name="Triage Agent",
    instructions="Route the user to the correct agent.",
    handoffs=[shopping_agent, support_agent],
)

output = Runner.run_sync(
    starting_agent=triage_agent,
    input="What shoes might work best with my outfit so far?",
)
```

OpenAI video on Vimeo

The Agents SDK is suitable for various real-world applications, including customer support automation, multi-step research, content generation, code review, and sales prospecting. For instance, [Coinbase](#) used the Agents SDK to quickly prototype and deploy AgentKit, a toolkit enabling AI agents to interact seamlessly with crypto wallets and various on-chain activities. In just a few hours, Coinbase integrated custom actions from their Developer Platform SDK into a fully functional agent. AgentKit's streamlined architecture simplified the process of adding new agent actions, letting developers focus more on meaningful integrations and less on navigating complex agent setups.

In a couple of days, [Box](#) was able to quickly create agents that leverage web search and the Agents SDK to enable enterprises to search, query, and extract insights from unstructured data stored within Box and public internet sources. This approach allows customers to not only access the latest information, but also search their internal, proprietary data in a safe and secure way that obeys their internal permissions and security policies. For example, a financial services firm can build a custom agent that calls on the Box AI agent to integrate their internal market analysis stored in Box with real-time news and economic data from the web, providing their analysts with a comprehensive view for investment decisions.

The Agents SDK works with the Responses API and Chat Completions API. The SDK will also work with models from other providers, as long as they provide a Chat Completions style API endpoint. Developers can immediately integrate it into their Python codebases, with Node.js support coming soon. Learn more in our [docs](#).

In designing the Agents SDK, our team was inspired by the excellent work of others in the community including [Pydantic](#), [Griffe](#) and [MkDocs](#). We're committed to continuing to build the Agents SDK as an open source framework so others in the community can expand on our approach.

What's next: building the platform for agents

We believe agents will soon become integral to the workforce, significantly enhancing productivity across industries. As companies increasingly seek to leverage AI for complex tasks, we're committed to providing the building blocks that enable developers and enterprises to effectively create autonomous systems that deliver real-world impact.

With today's releases, we're introducing the first building blocks to empower developers and enterprises to more easily build, deploy, and scale reliable, high-performing AI agents. As model capabilities become more and more agentic, we'll continue investing in deeper integrations across our APIs and new tools to help deploy, evaluate, and optimize agents in production. Our goal is to give developers a seamless platform experience for building agents that can help with a variety of tasks across any industry. We're excited to see what developers build next. To get started, explore our [docs](#) and stay tuned for more updates soon.

New tools for building agents with the API

